Guía de Buenas Prácticas y Flujo de Trabajo en Git y GitHub

Estándares y procedimientos para el desarrollo colaborativo de software.

1. Regla de Oro: La rama `main` no se toca directamente

La rama principal (main o master) representa el código en producción o listo para despliegue. Debe mantenerse siempre estable, funcional y libre de errores conocidos.

- **Prohibido hacer commits directos en `main`**: Ningún integrante del equipo debe realizar cambios directos sobre la rama principal.
- **Uso de Ramas de Tareas (`feature branches`)**: Todo cambio, corrección de errores o nueva funcionalidad debe desarrollarse en una rama independiente creada a partir de `main`.
- **Nomenclatura recomendada**:
    - Funcionalidades: `feature/nombre-tarea` (ej. `feature/login-jwt`)
    - Corrección de errores: `bugfix/descripcion-error` (ej. `bugfix/fix-header-padding`)
    - Tareas de mantenimiento: `chore/actualizacion-dependencias`

2. Paso a Paso del Flujo de Trabajo

Paso A: Crear una nueva rama desde `main` actualizada

Antes de iniciar cualquier trabajo, es fundamental asegurarse de contar con la versión más reciente del proyecto para evitar conflictos de integración futuros.

1. Posicionarse en la rama principal:
   `git checkout main`
2. Descargar e integrar las últimas actualizaciones del repositorio remoto:
   `git pull origin main`
3. Crear y cambiar a la nueva rama de trabajo:
   `git checkout -b feature/nombre-tarea`

Paso B: Trabajar y hacer commits pequeños y frecuentes

Mantener un historial de cambios granular facilita la revisión del código y la localización de posibles fallos.

1. Preparar los archivos modificados para el commit:
   `git add .`

2. Registrar los cambios con un mensaje claro, descriptivo y en presente imperativo:
   `git commit -m "Añade validación de formato en el formulario de registro"`

Paso C: Subir la rama a GitHub

Una vez completada la tarea (o para guardar un respaldo en la nube), se debe enviar la rama local al repositorio remoto.

1. Publicar la rama en el servidor remoto:
   `git push origin feature/nombre-tarea`

3. Pull Requests (PR) y Revisión de Código

El proceso de Pull Request es la herramienta principal para garantizar la calidad del código antes de su incorporación a la rama principal.

Pasos en GitHub para gestionar una PR:

1. **Abrir la PR**: Navegar al repositorio en GitHub y hacer clic en el botón **Compare & pull request** que aparece al detectar la nueva rama subida.
2. **Documentar el cambio**:
   - Escribir un título claro y una descripción detallada que explique el *qué* y el *porqué* de los cambios.
   - Vincular el ticket o issue correspondiente si aplica.
3. **Asignar Revisores**:
   - En la sección lateral derecha (**Reviewers**), seleccionar a uno o más compañeros de equipo para auditar el código.
4. **Proceso de Revisión**:
   - El revisor examina el código, deja comentarios constructivos o solicita ajustes si es necesario.
   - El autor realiza los cambios solicitados en su entorno local y vuelve a hacer `git push` a la misma rama (la PR se actualizará automáticamente).
5. **Merge y Limpieza**:
   - Una vez aprobada la PR por al menos un revisor y superadas las pruebas automatizadas, presionar el botón **Merge pull request**.
   - Eliminar la rama remota tras el merge utilizando la opción que ofrece la interfaz de GitHub.

4. Sincronización Diaria

Durante el desarrollo de tareas extensas, la rama `main` puede recibir cambios introducidos por otros miembros del equipo. Es imprescindible mantener la rama de trabajo actualizada para prevenir conflictos masivos al momento de integrar.

Actualizar la rama de trabajo local:

1. Guardar o hacer commit de los cambios actuales en la rama de trabajo.
2. Descargar las últimas novedades de la rama principal:
   ```
   git checkout main
   git pull origin main
   ```
3. Regresar a la rama de trabajo e integrar los cambios de `main`:
   ```
   git checkout feature/tu-tarea
   git merge main
   ```
4. Resolver conflictos de código en caso de que existan, hacer commit de la resolución y continuar con el desarrollo.

5. Resumen de Buenas Prácticas

Práctica	Descripción	Beneficio principal
Commits pequeños	Agrupar cambios lógicos y concretos en cada commit.	Facilita el rastreo de errores y permite revertir cambios específicos sin afectar otras partes.
Mensajes claros	Usar el formato `<tipo>: <descripción>` (ej. `feat: agregar boton`).	Mejora la legibilidad del historial y automatiza la generación de notas de versión.
Uso de `.gitignore`	Configurar adecuadamente los archivos excluidos (dependencias, credenciales, temporales).	Evita subir archivos innecesarios o sensibles al repositorio central.
Comunicación previa	Notificar al equipo antes de asumir tareas o realizar cambios estructurales.	Evita la duplicación de esfuerzos y minimiza la aparición de conflictos de código complejos.